

En JavaScript, un **array** (arreglo) es una estructura de datos que te permite almacenar múltiples valores en una sola variable. Aquí te explico cómo **definirlo y manipularlo** con ejemplos claros.

📌 1. Cómo definir un array

✅ Forma más común (con corchetes)

```
1 let frutas = ["manzana", "banana", "naranja"];
```

✅ Usando el constructor Array

```
1 let numeros = new Array(1, 2, 3, 4);
```

📌 2. Acceder a elementos

Los arrays usan índices (empiezan en 0):

```
1 let frutas = ["manzana", "banana", "naranja"];
2
3 console.log(frutas[0]); // "manzana"
4 console.log(frutas[2]); // "naranja"
```

📌 3. Modificar elementos

```
1 frutas[1] = "pera";
2 console.log(frutas); // ["manzana", "pera", "naranja"]
```

📌 4. Agregar elementos

✅ Al final (push)

```
1 frutas.push("kiwi");
```

✅ Al inicio (unshift)

```
1 frutas.unshift("uva");
```

📌 5. Eliminar elementos

✅ Último (pop)

```
1 frutas.pop();
```

✅ Primero (shift)

```
1 frutas.shift();
```

✅ Eliminar por posición (splice)

```
1 frutas.splice(1, 1); // elimina 1 elemento desde la posición 1
```

📌 6. Recorrer un array

✅ Con for

```
1 for (let i = 0; i < frutas.length; i++) {
2   console.log(frutas[i]);
3 }
```

✅ Con forEach

```
1 frutas.forEach(fruta => {
2   console.log(fruta);
3 });
```

📌 7. Métodos útiles

✅ map (transformar datos)

```
1 let numeros = [1, 2, 3];
2
3 let doble = numeros.map(n => n * 2);
4 console.log(doble); // [2, 4, 6]
```

✅ filter (filtrar elementos)

```
1 let numeros = [1, 2, 3, 4];
2
3 let pares = numeros.filter(n => n % 2 === 0);
4 console.log(pares); // [2, 4]
```

✅ find (buscar un elemento)

```
1 let numeros = [1, 2, 3, 4];
2
3 let encontrado = numeros.find(n => n > 2);
4 console.log(encontrado); // 3
```

📌 8. Propiedad importante

✅ Longitud del array

```
1 let frutas = ["manzana", "banana"];
2 console.log(frutas.length); // 2
```

✅ Resumen

- **♦** `[]` → crear array
- **♦** `push`, `pop` → agregar/quitar al final
- **♦** `shift`, `unshift` → inicio
- **♦** `splice` → eliminar/modificar posiciones
- **♦** `map`, `filter`, `find` → manipulación avanzada